



Matt Dailey

@reactiverobot

Wed Aug 26 15:39:14 +0000 2026

How I Design with AI.

As an engineer who is not a designer and hates slop.

How I Design with AI.

As an engineer who is not a designer and hates slop. Every landing page, app and tui look the same. They're slop and most of them are incomprehensible. Here's how I de-slop my product design. 1.

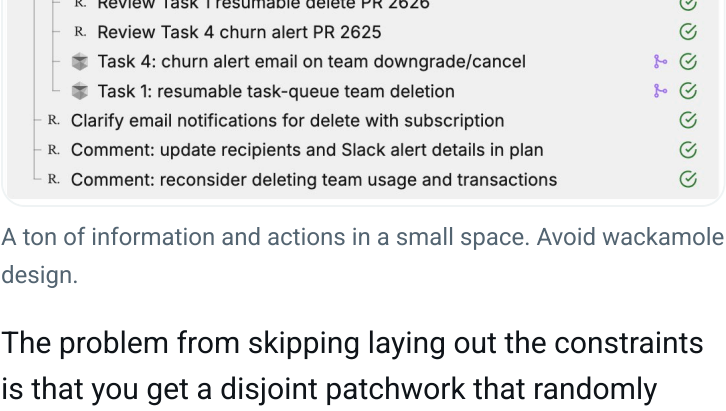
As an engineer who is not a designer and hates slop. Every landing page, app and tui look the same. They're slop and most of them are incomprehensible. Here's how I de-slop my product design.

1. Always consider the whole.

The design process is roughly 3 steps:

- Lay out all the constraints you are designing for.
- Consider an array of solutions that satisfy those constraints.
- If you realize a constraint must be added or one can be removed, go to #1.

This is from Notes on the Synthesis of Form by Christopher Alexander. It's a pleasingly rigorous exploration of the design process.

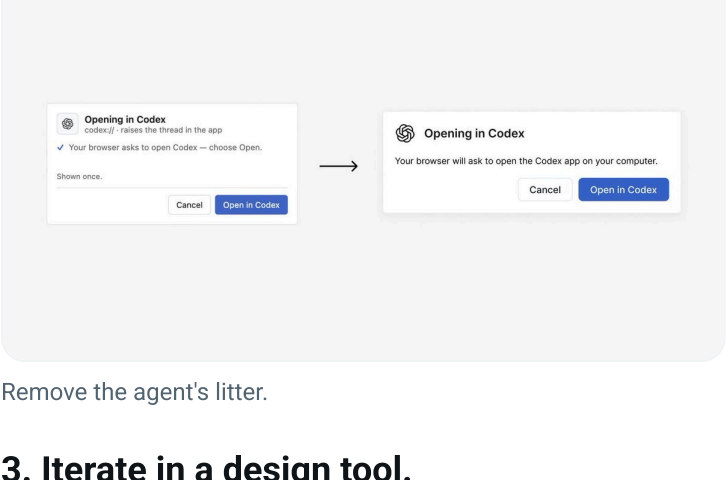


Notes on the Synthesis of Form, a very good book.

Constraints can take many forms. They could be font and sizing rules, workflows you must support or business-logic states. The important thing is that you decide the constraints.

The common problem is skipping Step 3 and playing design wackamole.

A user gets confused and it's natural to jump into solution mode. Our team gets nerd sniped by this all the time. Especially with the left sidebar because it packs so much information into a small space. The temptation is to spot-fix but that road leads to ruin.



A ton of information and actions in a small space. Avoid wackamole design.

The problem from skipping laying out the constraints is that you get a disjoint patchwork that randomly prioritizes some interactions over others. AI exacerbates the draw to wackamole design. It encourages prompting "Make X more prominent" or "Add an affordance to do Y". In the end, more users are confused.

When feedback arrives, evaluate if it changes your design constraints before jumping to solutions. The way we handle this is keeping a document tracking papercuts and annoyances. We move fast on obvious fixes but track the minor ones so that when it comes time to redesign, we have a collection to address cohesively. The important thing is that we avoid being overly reactive and creating more mess.

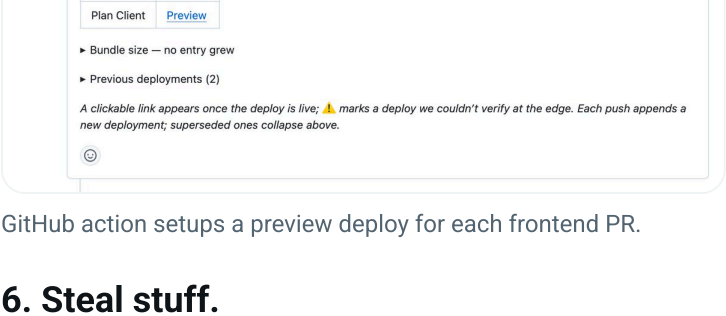
2. Remove stuff.

Agents love to add stuff. Your job is to remove the unnecessary parts.

This should be very familiar because agents do exactly the same thing in code and plans. They love belt-and-suspenders, wrapping an extra try-catch or re-implementing the same utility over and over.

In UI designs, agents do the same thing. They love adding extra copy, lines and icons. You end up with real designs that look better than most engineers would create by hand but are actually kind of bad.

An easy step is just look at every element of a design and for every element ask "Do I actually need that?"



Remove the agent's litter.

3. Iterate in a design tool.

You should not be iterating on design in the product. Use a tool that gives you fine control and lets you iterate quickly with minimal extra context.

Prototype gravity is the silent killer. This is when you have the agent build the first version in your code base then it feels easier to just refine that instead of exploring other options. Designing in your real codebase also forces the agent to build a version that grafts onto your real

Figma is still the GOAT and the AI integration is getting better every week. Cursor Design Mode, Claude Design, a bunch of new startups and even HTML prototypes are great too.

Just please use a tool meant for design and have AI generate 3-4 variants of everything.



Figma is still the GOAT of design iteration.

4. Use components and libraries.

This one is probably obvious to most engineers but it's very important so I'll say it quickly.

Separate views and logic. Create reusable components. It's not hard and it pays dividends. Your app will be visually cohesive rather than a patchwork of re-implemented buttons.

To do this at Ref, we maintain a /showcase page. We have agents build the UI component there first to play with them before connecting to the main app.

The /showcase page with all of Ref's components.

5. Use preview deploys.

The best way to evaluate a design is with real data. Preview deploys allow you to try the new designs with your real backend.

It's important to remember, there will always be some polish and re-work necessary. Even if the agent builds exactly what you told it, you may still hold it in your hands with real data and realize it's not right.

For a large feature that involves frontend and backend, preview deploys can be tricky. At Ref we solve this by separating frontend and backend PRs. Backend changes can be verified with unit and integration tests. Frontend changes require human verification and preview deploys make it easy to share a link.

GitHub action setups a preview deploy for each frontend PR.

6. Steal stuff.

Most UX problems have been solved already and you should be taking pieces and putting them together. Spend some time looking at products solving a similar problems or communicating similar ideas.

Every cracked designer I've worked with starts every single project by pulling together a bunch of screenshots. You should do the same, it makes amazing context to send to your agent.

Landing page inspiration gathered by an very good actual designer.

7. Explore your taste.

This is the fun part!

Taste is reflecting on your own reaction to something. And my apologies to Kyle Chayka but reflecting on one's experience is not proprietary to parties in Brooklyn lofts. It's necessary to create and not create slop.

Product engineers are excellent at identifying when a design doesn't work but have a hard time knowing how to fix it. What engineers lack is the solution library from experience to pull from. Building that just takes reps trying things and reflecting. It's fun to play and explore. But it can also feel pretty brutal because its a deluge of critique until something is good enough.

At Ref we don't have a full-time designer so we resort to an agricultural threshing approach to refining our taste. We throw a design in the middle and beat it with sticks until we feel good about it.

The Ref team beating a design into submission.

That's all I've got. GLHF.

Source / Copyright attribution

Author: @reactiverobot (Matt Dailey)

Post: <https://x.com/reactiverobot/status/2092638003789439075>

Fetched: 2026-09-14

Images and text copyright remain with the original author / X (Twitter)

r).

Archived for personal reference; do not redistribute without permission.